

ニューラルネットワークの最適化② 誤差逆伝播

ニューラルネット > 最適化



基本的には勾配法ベースの手法で最適化を行う

- 基本的に大域的最適解を求める解の公式はないので、勾配ベースの方法で最適化

$$\hat{\theta} \leftarrow \hat{\theta} - \eta \cdot \nabla L(\hat{\theta})$$

- これには、目的関数のパラメーターに関する勾配を計算する必要がある
 - 例) ソフトマックス + 負の対数尤度の場合

$$\nabla \ell((\mathbf{x}, y), g_{\theta}) = -\nabla s_{\theta}(\mathbf{x})[y] + \sum_k g_{\theta}(\mathbf{x})[k] \cdot \nabla s_{\theta}(\mathbf{x})[k]$$

- スコアリングモデルの出力成分 $s_{\theta}(\mathbf{x})[k]$ の勾配 $\nabla s_{\theta}(\mathbf{x})[k]$ を計算する必要がある

ニューラルネット > 最適化 > 勾配計算



パラメーターが複雑に絡み合っているので、微分を愚直に計算したくはない

- 例えば3層のモデル ($\psi_1 = \psi_2 = \text{ReLU}$, $\psi_3 = \text{Softmax}$ とする) なら,

$$g_{\theta}(\mathbf{x}) = \psi_3(W_3\psi_2(W_2\psi_1(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3)$$

- 目的関数

$$L(\theta) := -\frac{1}{n} \sum_{i=1}^n \log g_{\theta}(\mathbf{x})[y_i]$$

- このとき、現在のパラメーターの具体的な値 $\theta = (W_1, \mathbf{b}_1, W_2, \mathbf{b}_2, W_3, \mathbf{b}_3)$ における,
 $L(\theta)$ の $W_1, \mathbf{b}_1, W_2, \mathbf{b}_2, W_3, \mathbf{b}_3$ のそれぞれに関する微分 (勾配) を計算せよ.

ニューラルネット＞最適化＞勾配計算の例

手計算できる例で説明する

- 1次元特徴量での回帰を考える.
 - モデルは次の順伝播型NNとする

$$g_{(w,b)}(x) = \sigma(wx + b)$$

- データは1点 (x, y) だけとして, 目的関数は二乗誤差とする

$$L(w, b) = (y - g_{(w,b)}(x))^2$$

- 勾配 $\nabla L(w, b)$ を計算せよ.

ニューラルネット > 最適化 > 勾配計算の例 > 手計算



試しに $L(w, b) = (y - \sigma(wx + b))^2$ の w による微分の計算手順を考えてみる

- まず外側に2乗があるので, $(x^2)' = 2x$ を使って,

$$(y - \sigma(wx + b))^2 \rightsquigarrow 2(y - \sigma(wx + b)) \cdot (\text{中身の}(y - \sigma(wx + b))\text{の微分})$$

- 続いて中身の $(y - \sigma(wx + b))$ を w で微分するから,

$$(y - \sigma(wx + b)) \rightsquigarrow -\sigma'(wx + b) \cdot x \text{ のようになって,}$$

- 残る $\sigma'(wx + b)$ の部分は, 確か $\sigma'(u) = \sigma(u)(1 - \sigma(u))$ だったから, まとめると

$$\frac{\partial L(w, b)}{\partial w} = -2(y - \sigma(wx + b))\sigma(wx + b)(1 - \sigma(wx + b))x$$

- b による微分も同じように, …… (少し大変)

ニューラルネット > 最適化



パラメーターが多段階で変換に絡むため、勾配の式は複雑になる

- しかし、実はシステムチックに勾配を計算する方法がある
 - それは、入力 x から損失値が出てくるまでの計算順をまず考えて、その後に、それを遡りながら微分を計算するという方法

ニューラルネット > 最適化 > 機械的に勾配を計算する方法

まずは、入力から損失の値が出てくるまでの計算手順を図にする

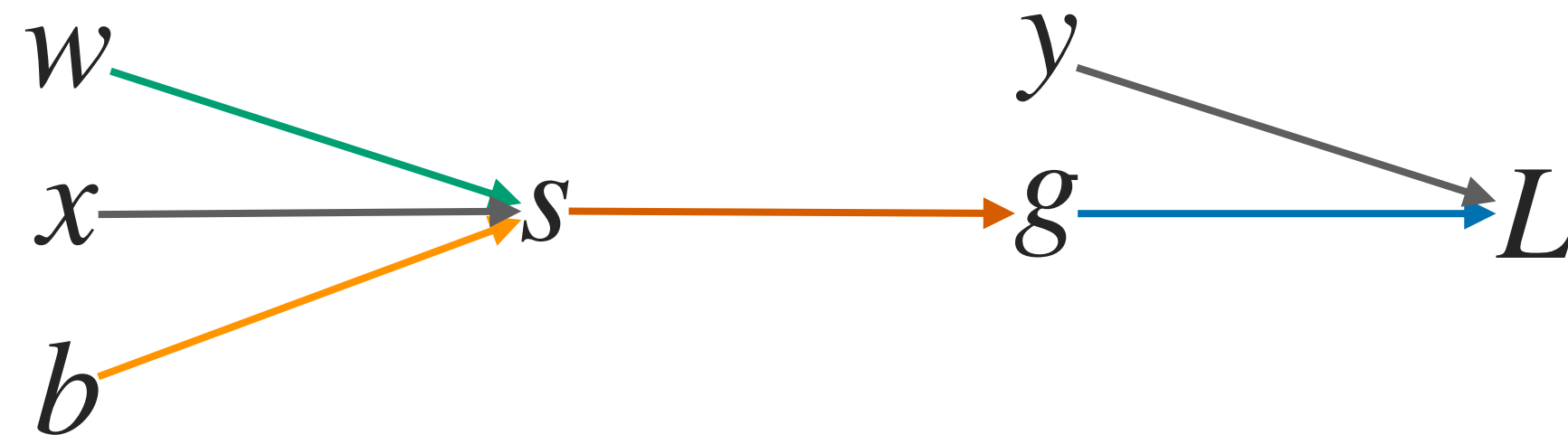
- モデルと目的関数の計算式は次のとおりだが……

$$L(w, b) = (y - g_{(w,b)}(x))^2 \quad (\text{ここで } g_{(w,b)}(x) = \sigma(wx + b))$$

- 途中の計算結果の値にも次のように記号を割り振っておいて……

$s = wx + b$ を計算し, $g = \sigma(s)$ を計算し, $L = (y - g)^2$ を計算

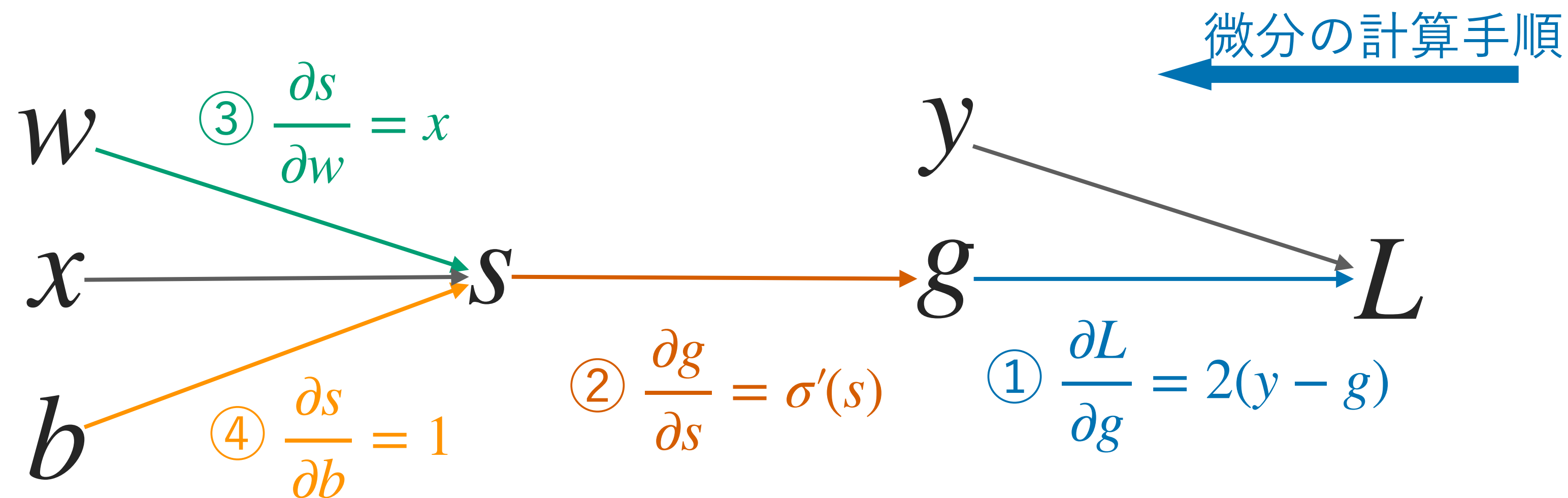
- 計算手順の通りに、入力やパラメーターから損失値 L が決まるまでを図にする



ニューラルネット > 最適化 > 機械的に勾配を計算する方法

この計算順を描いた図の「矢印」を逆順にたどりながら微分を計算する

- 例えば $g \rightarrow L$ の矢印では, L の g での偏微分 $\frac{\partial L}{\partial g} = 2(y - g)$ を計算 (他も同様)



- 最後に掛け算で $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial g} \frac{\partial g}{\partial s} \frac{\partial s}{\partial w}$, $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial g} \frac{\partial g}{\partial s} \frac{\partial s}{\partial b}$ を得る ($\frac{\partial L}{\partial g} \frac{\partial g}{\partial s}$ は再利用)

ニューラルネット＞最適化＞計算グラフと自動微分



もっと複雑なニューラルネットでも全く同じことができる

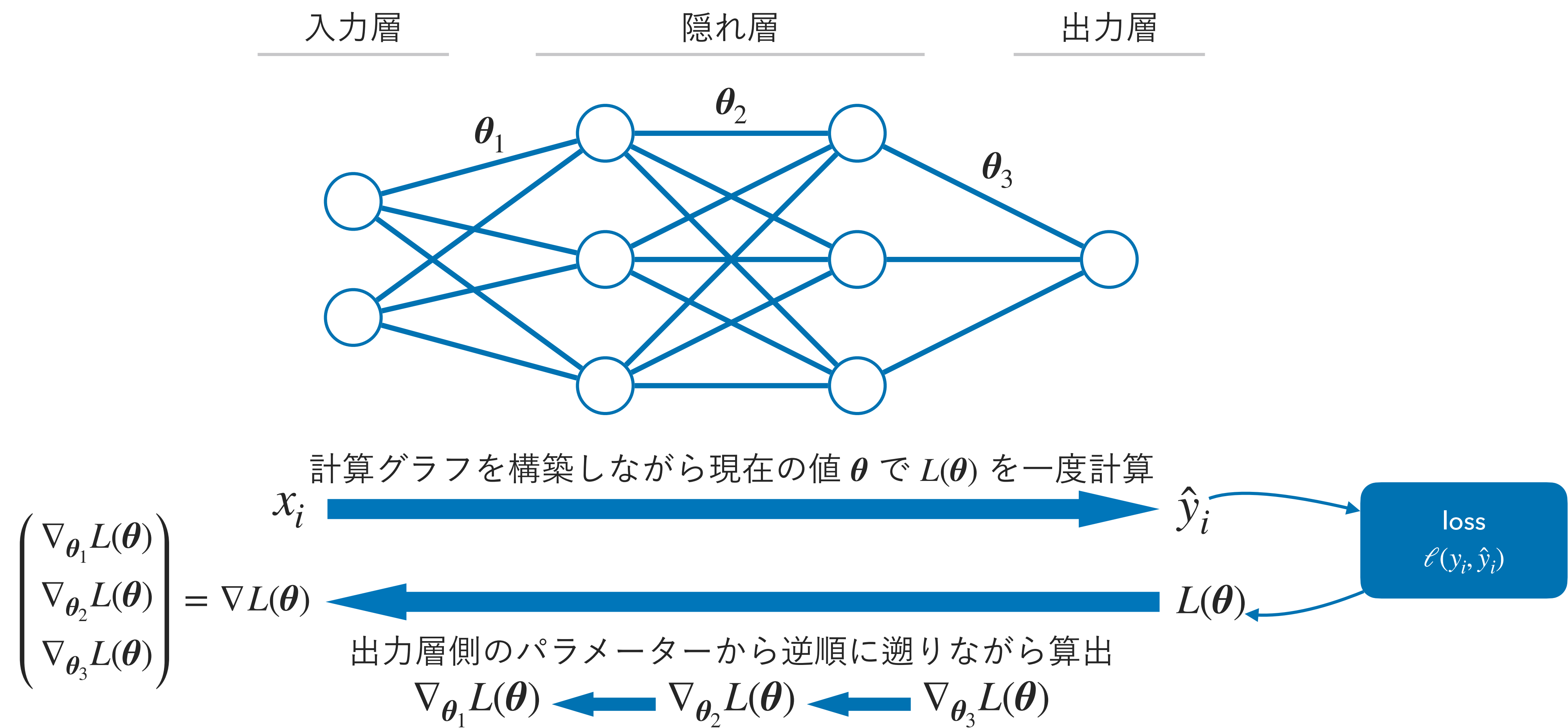
- このようにパラメーターやデータの変数を，計算の依存関係に従って矢印で繋いだ図を，**計算グラフ**（computation graph）という。
- 一般の計算グラフでも，全く同じように，システムチックに勾配計算をできる
 - これを**自動微分**（automatic differentiation）という
 - 全結合層である必要もなく，どんなアーキテクチャでも勾配を計算できる
 - 今では「微分可能な計算グラフ」の全般がニューラルネットと呼ばれる

【補足】どんな構造でも勾配をシステムチックに導出できるとはいえ，その勾配の計算効率が良いかどうかは別の話。

【補足】計算グラフとは，変数を頂点とする非巡回有向グラフで，各頂点に親の変数を引数とする関数が割り当てられているもの。なお，非巡回有向グラフとは，頂点と有向辺の集まりで，周回できる道がないもののこと。

ニューラルネット > 最適化 > 誤差逆伝播法 (Backpropagation)

自動微分によって $\nabla L(\theta)$ を出力層側から順に遡りながら算出



【用語】 損失関数 ≡ 誤差の情報を、入力側の層にめがけて伝えているように見えるので、誤差逆伝播法という。

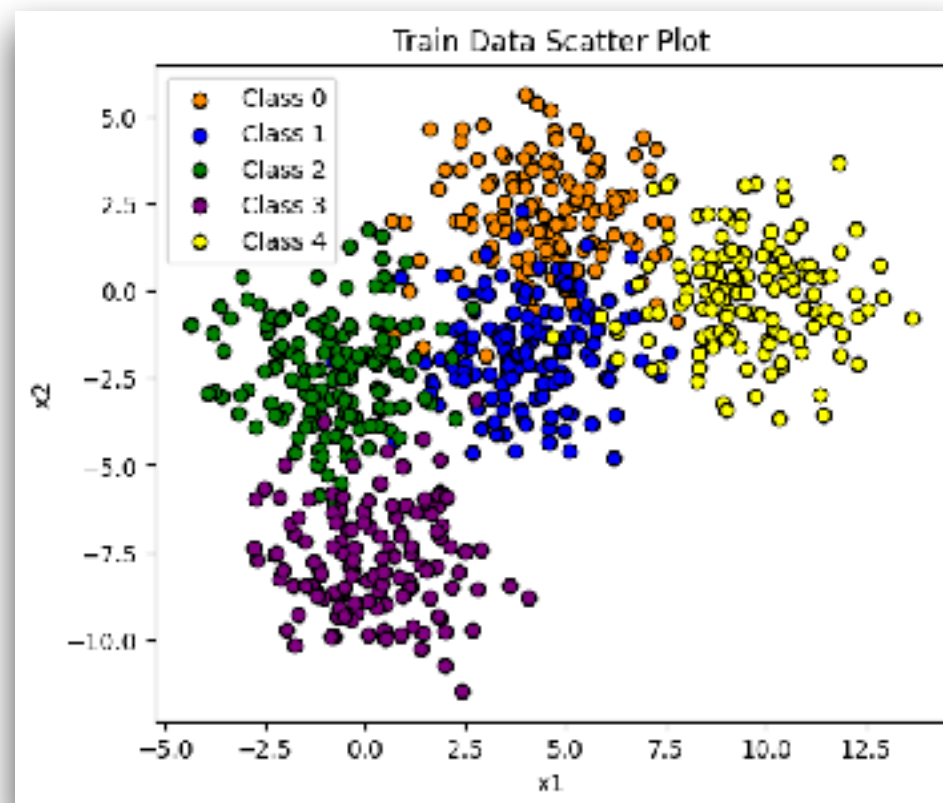
ニューラルネット＞実験＞設定

多クラス分類をニューラルネットで学習

タスク・学習戦略

5クラス分類・教師付き学習

特徴量	ラベル
x_1	y_1
x_2	y_2
$\vdots$	$\vdots$
$\{(X_i, Y_i)\}_{i=1}^n \stackrel{\text{i.i.d.}}{\sim} p$	



訓練データ

モデル

$g_{\theta}(x)$: 順伝播型NN

- 各層のユニット数は以下.
 - ▶ 入力層: 2 (入力次元)
 - ▶ 隠れ層①: 64
 - ▶ 隠れ層②: 32
 - ▶ 出力層: 5 (クラス数)
- 活性化関数はReLU, 最終層のみSoftmax.

目的関数

負の対数尤度

$$L(\theta) := \frac{1}{n} \sum_{i=1}^n \ell((x_i, y_i), g_{\theta})$$

- モデル選択基準は検証損失値 (早期停止 = 学習エポック数のみ選択)
- L2正則化はなし

最適化手法

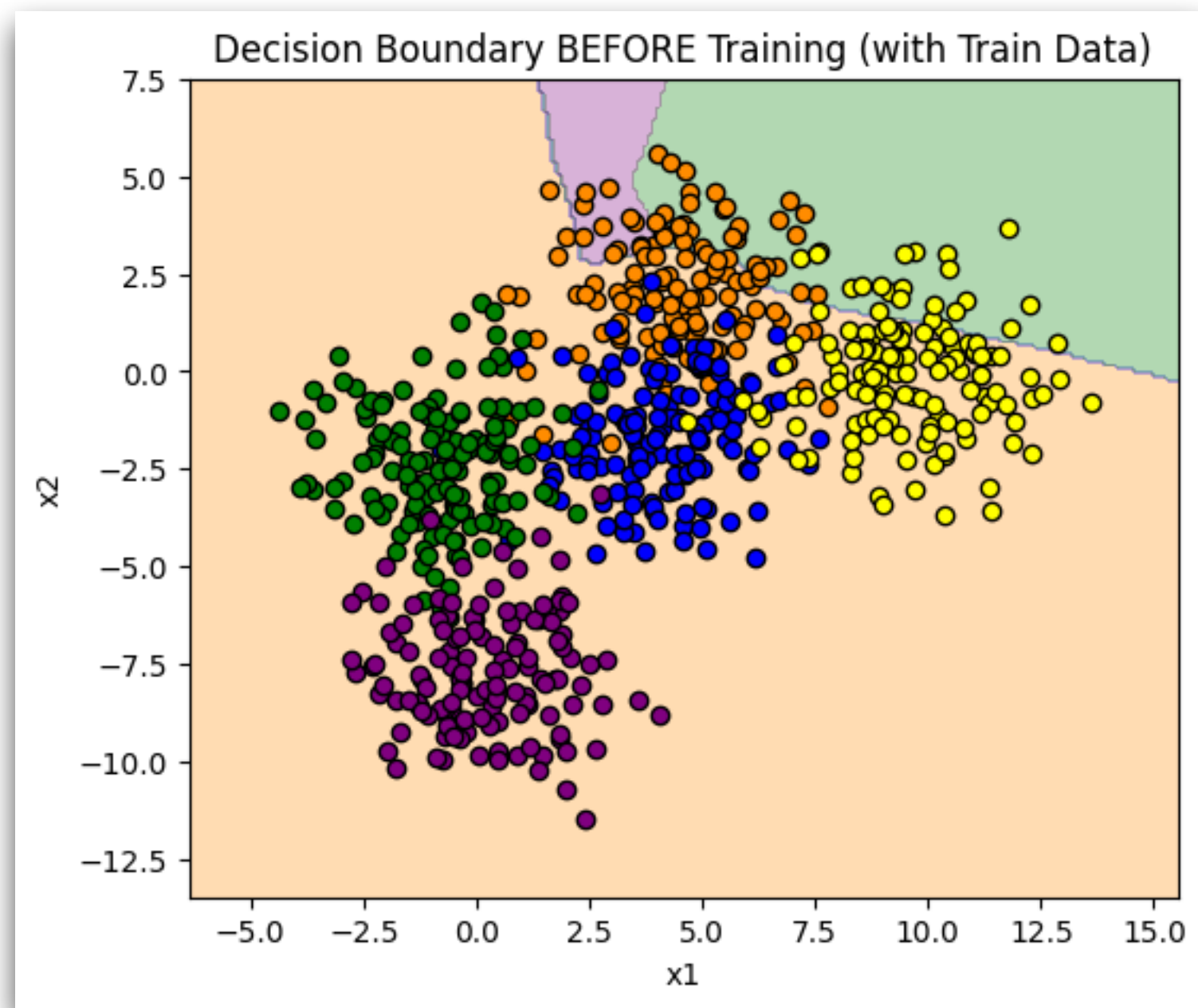
Adam

(慣性項を用いる確率的勾配法の一つ)

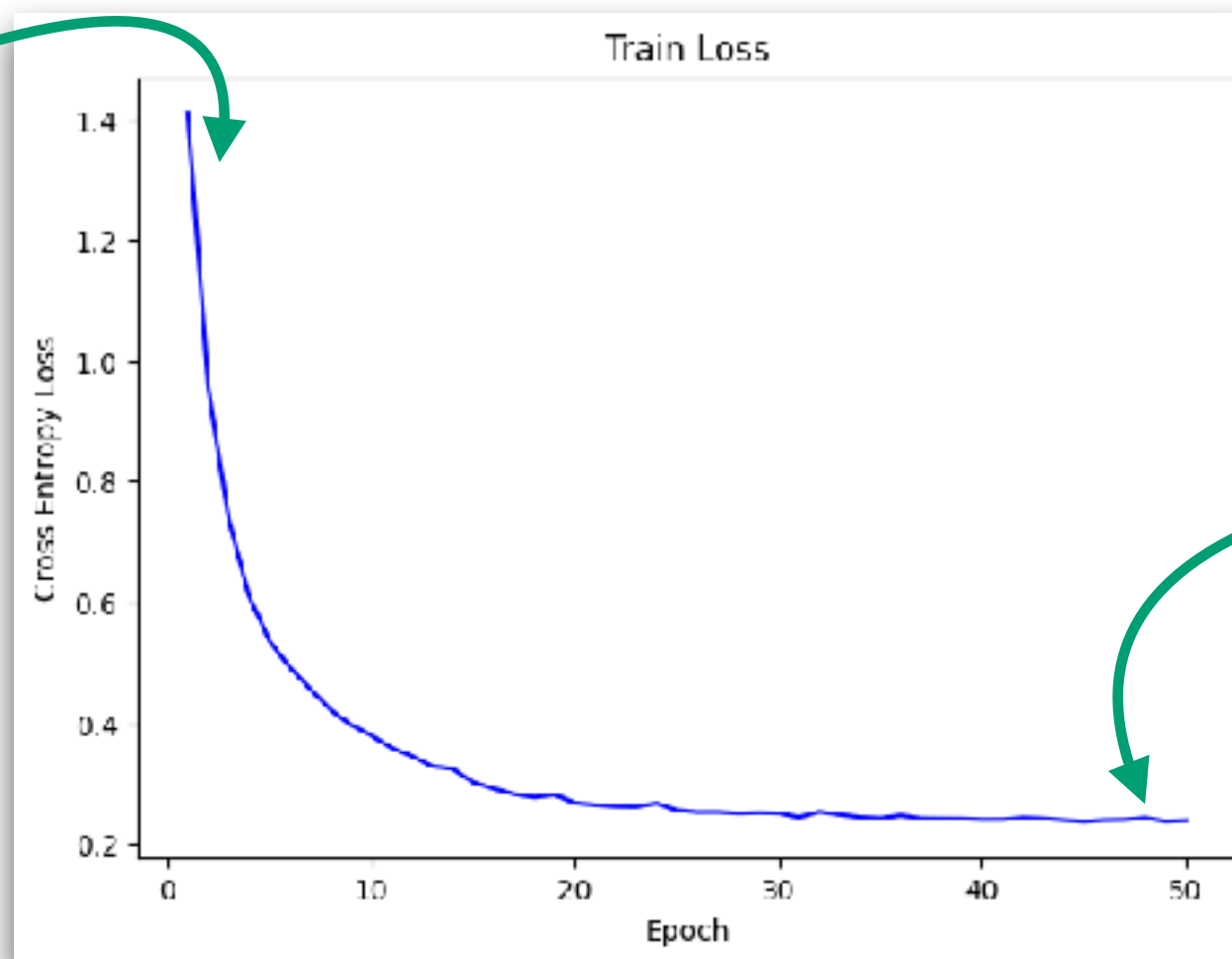
- バッチサイズは 32
- 学習率 η は 10^{-3}
- エポック数上限 100
- 早期停止 (直近10エポックの改善幅が0以下で停止)
- 初期値 $\hat{\theta}$ はHeの初期化

ニューラルネット＞実験＞結果

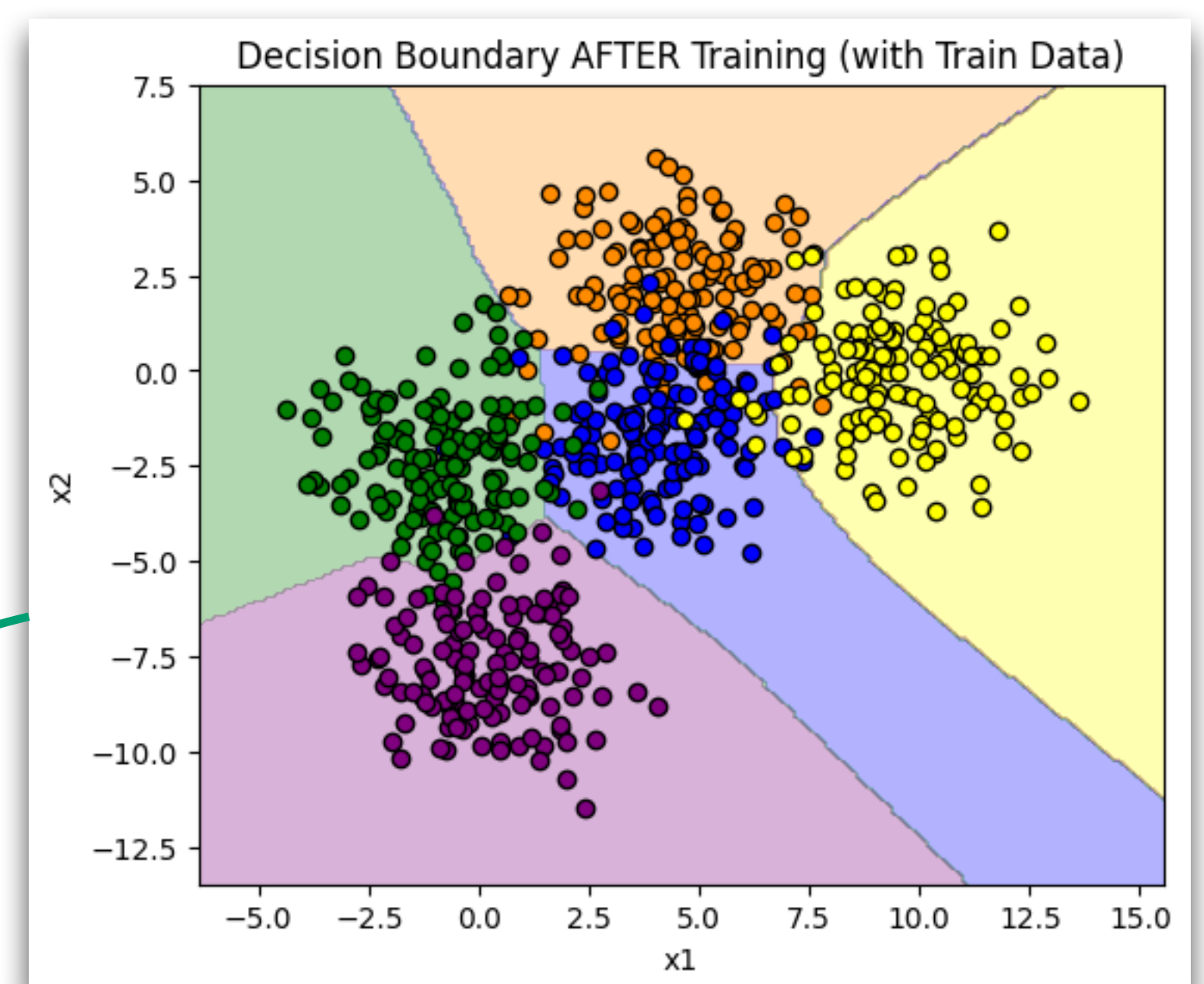
訓練前の初期化した時点での予測器と，訓練後の予測器



訓練前のモデルと訓練標本



訓練誤差

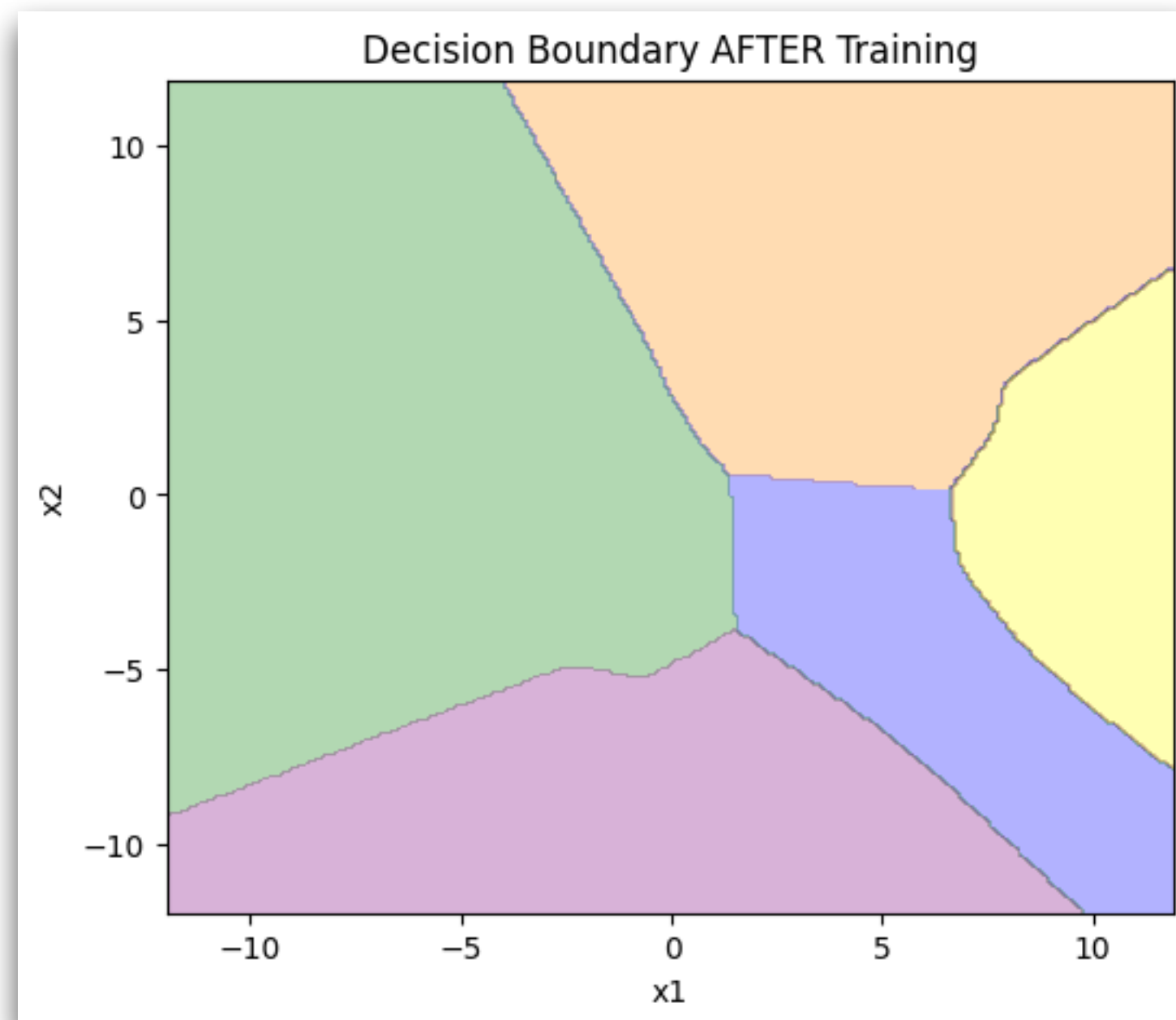


訓練後のモデルと訓練標本

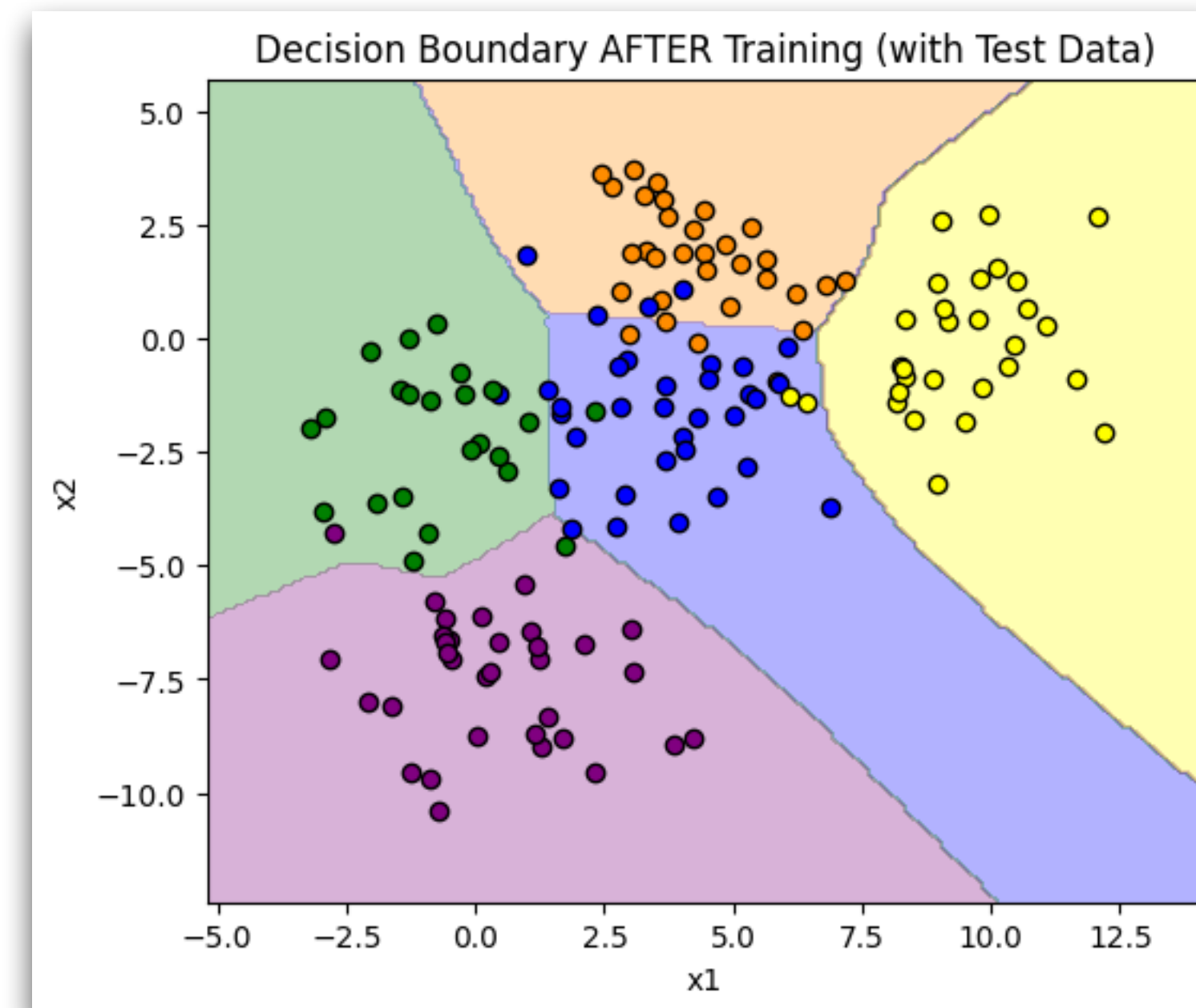
ニューラルネット > 実験 > 学習過程

予測器としても良さそうなものになっている

- 学習後のモデルに決定規則を合成した予測器 $f_{\hat{\theta}}(x) := \tau(g_{\hat{\theta}}(x))$ の様子



分類器としての出力
(スコア最大値を与えるラベルを予測)



評価用データでの予測と正解ラベル

ニューラルネット > まとめ

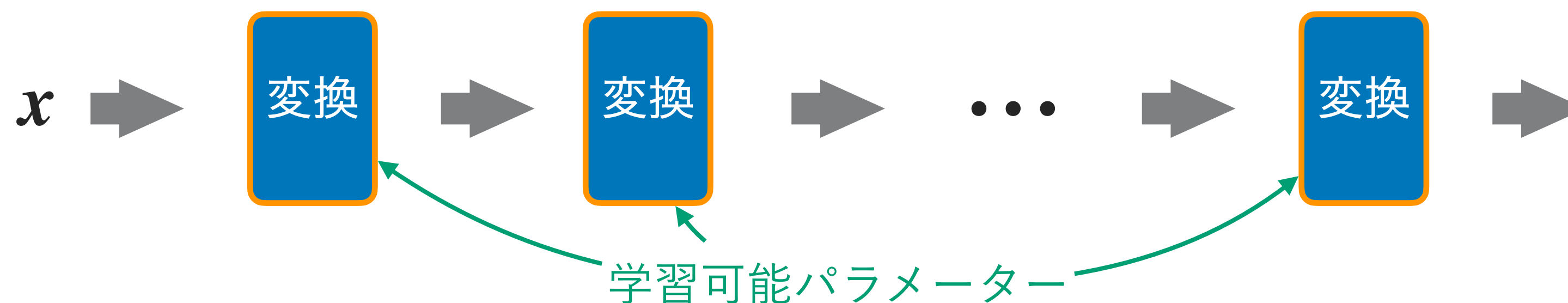


ニューラルネットは階層性と構造の自由度がポイント。最適化は大変。

- ニューラルネットワークというモデルは、非線形モデルの主要なものの一つであり、実用的にも広く使われている。
 - 順伝播型NNは、線型変換→非線型変換→……を積み重ねて複雑な関数を表現
 - 自由度の高さを活かして様々なアーキテクチャーのNNが開発されている
- 最適化は誤差逆伝播法の考え方を使えば、複雑なアーキテクチャーのNNでも汎用的に勾配を計算できて、勾配ベースの最適化手法が適用できる
 - ただし、ニューラルネットの最適化は高難易度で、様々な工夫を要する
 - 実際には、最新時点で評判の良い最適化手法を調べて使うことから

これまでのあらすじ

- ニューラルネットワークという柔軟な道具を手に入れた



- 最終層の手前までを特徴写像 ϕ だと思えば、特徴量変換も含めて一気通貫で学習可能なモデルともいえる
- 対して線型モデルでは、 ϕ で特徴量に変換 $\rightarrow \theta^\top$ を掛けて出力 $\rightarrow \theta^\top \phi(x)$
- これは人類にとって重要な道具になった。一体なぜ重要になったのか？